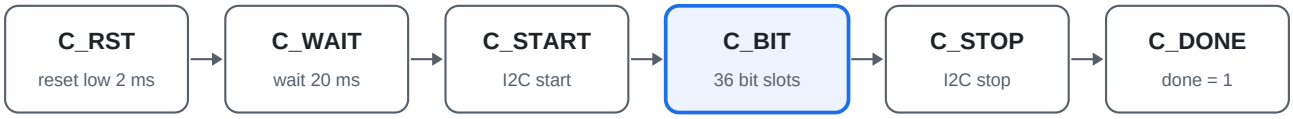
cam_config.v, explained part by part

Camera reset and I2C set-up for the TRDB-D5M camera, DE2-115 live JPEG compression project. Prepared by Dev Vijay for the project review.

1. What the module does

cam_config prepares the camera before any picture is taken. It holds the camera in reset, lets it start up, then sends it **15 settings** over **I2C**, a two-wire bus (SCLK = clock, SDATA = data). When all 15 are sent it raises **done**, which lights LEDG1 and lets the top module start capturing. The whole sequence takes about **29 ms** and runs again whenever KEY0 (reset) is pressed.

Sequence after power-up or KEY0



C_START -> C_BIT -> C_STOP repeats 15 times, once per register

What the 15 settings do

Register	Value	Effect
0x20	0xC000	Read mode: mirror rows and columns so the picture is upright
0x09	exposure	Shutter width (exposure time) in rows: 256 to 4096, from SW16 to SW13
0x05, 0x06	0, 25	Horizontal and vertical blanking (gaps between lines and frames)
0x0A	0x8000	Invert the pixel clock so data is stable on the rising edge we sample on
0x2B to 0x2E	0x13, 0x9A, 0x19C, 0x13	Gains for green 1, blue, red, green 2 (colour balance)
0x01, 0x02	386, 672	First row and column: centre a 1280 x 1280 window on the 2592 x 1944 sensor
0x03, 0x04	1279, 1279	Window height and width minus 1 (1280 x 1280)
0x22, 0x23	0x0001	Skip every 2nd row and column: 1280 x 1280 window -> 640 x 640 output

cam_capture then merges each 2 x 2 group into one pixel, so 640 x 640 becomes the 320 x 320 picture the compression engine expects.

2. The code, part by part

Part A - Header and ports (lines 1-25)

Lines 15-25

```
module cam_config (
    input  wire       clk,            // 25 MHz
    input  wire       reset,          // KEY0: reset camera and send settings again
    input  wire [3:0] exposure_sel,   // SW16..SW13: 0 = darkest ... 15 = brightest
    output reg        cam_reset_n,    // camera reset (active low)
    output reg        sclk,           // I2C clock
    inout  wire       sdata,          // I2C data (open drain)
    output reg        done            // 1 = camera is set up
);

    localparam NREG = 15;
```

- **clk** is the same 25 MHz clock used by the rest of the design.
- **exposure_sel** comes from SW16 to SW13; it is read only on reset, so change the switches then press KEY0.
- **cam_reset_n** is active low: 0 holds the camera in reset.
- **sdata** is **inout** because I2C data is open-drain: the FPGA either pulls it low or lets go of it.

Part B - The register list (lines 27-49)

Lines 27-49

```
    // ---------------- register list {register, value} ----------------
    function [23:0] entry;
        input [4:0]  i;
        input [15:0] expo;
        case (i)
            5'd0:  entry = {8'h20, 16'hC000};   // read mode: mirror rows + columns (picture upright)
            5'd1:  entry = {8'h09, expo};       // exposure time (shutter width, in rows)
            5'd2:  entry = {8'h05, 16'h0000};   // horizontal blanking (minimum)
            5'd3:  entry = {8'h06, 16'h0019};   // vertical blanking (25 rows)
            5'd4:  entry = {8'h0A, 16'h8000};   // invert pixel clock (data stable on rising edge)
            5'd5:  entry = {8'h2B, 16'h0013};   // green 1 gain
            5'd6:  entry = {8'h2C, 16'h009A};   // blue gain
            5'd7:  entry = {8'h2D, 16'h019C};   // red gain
            5'd8:  entry = {8'h2E, 16'h0013};   // green 2 gain
            5'd9:  entry = {8'h01, 16'd386};    // first row    (centre of the 1944-row sensor)
            5'd10: entry = {8'h02, 16'd672};    // first column (centre of the 2592-column sensor)
            5'd11: entry = {8'h03, 16'd1279};   // window height - 1  (1280 rows)
            5'd12: entry = {8'h04, 16'd1279};   // window width  - 1  (1280 columns)
            5'd13: entry = {8'h22, 16'h0001};   // rows:    skip 2x -> 640 rows out
            5'd14: entry = {8'h23, 16'h0001};   // columns: skip 2x -> 640 columns out
            default: entry = {8'h20, 16'hC000};
        endcase
    endfunction
```

- A Verilog **function** with a **case**: given index i (0..14) it returns 24 bits = 8-bit register address + 16-bit value.
- Quartus turns it into a small table (ROM) made of logic.
- The exposure value is passed in as **expo**, because it is not a constant: it depends on the switches.
- 16'd386 etc. are decimal constants; 16'hC000 is hexadecimal.

Part C - I2C timing and the open-drain data line (lines 51-57)

Lines 51-57

```
    // ---------------- I2C timing: 1 bit = 4 quarters of 63 clocks (~99 kHz) ----------------
    reg  [5:0] qdiv = 6'd0;
    wire       tick = (qdiv == 6'd62);
    always @(posedge clk) qdiv <= tick ? 6'd0 : qdiv + 6'd1;

    reg sda_o;
    assign sdata = sda_o ? 1'bz : 1'b0;          // open drain: release for 1, pull low for 0
```

- **qdiv** counts 0..62 and **tick** is 1 for one clock every 63 clocks.
- Every I2C bit is split into 4 **quarters**, one per tick: 4 x 63 = 252 clocks = 10.08 us per bit, so the I2C clock is about **99 kHz** (the standard rate is 100 kHz).
- **Open drain:** for a 1 the FPGA lets the line float (1'bz) and the pull-up resistor makes it high; for a 0 it pulls the line low. This lets the camera also drive the line to answer (acknowledge) without a short circuit. A weak pull-up is also enabled on the FPGA pin in cam_pins.tcl.

Part D - States and registers (lines 59-75)

Lines 59-75

```
    localparam C_RST = 3'd0, C_WAIT = 3'd1, C_START = 3'd2, C_BIT = 3'd3,
               C_STOP = 3'd4, C_DONE = 3'd5;

    reg [2:0]  st;
    reg [19:0] wcnt;
    reg [1:0]  q;           // quarter of the current bit (0..3)
    reg [4:0]  idx;         // which register is being written
    reg [1:0]  byte_i;      // byte 0..3 of the current write
    reg [3:0]  bit_i;       // bit 0..7, 8 = acknowledge slot
    reg [31:0] sh;          // bits still to send
    reg [15:0] exposure;

    initial begin
        st = C_RST; wcnt = 0; q = 0; idx = 0; byte_i = 0; bit_i = 0; sh = 0;
        cam_reset_n = 1'b0; sclk = 1'b1; sda_o = 1'b1; done = 1'b0;
        exposure = 16'h0100;
    end
```

Register	Purpose
st	current state (6 states)
wcnt	counts clocks for the 2 ms and 20 ms waits (20 bits, up to 1,048,575)
q	which quarter of the current I2C bit (0..3)
idx	which of the 15 registers is being written
byte_i, bit_i	byte 0..3 of the current write and bit 0..8 inside it (8 = acknowledge slot)
sh	32-bit shift register holding the bits still to send; the top bit sh[31] goes out next
exposure	the shutter-width value, calculated from the switches on reset

The **initial** block sets power-up values, so the sequence also runs right after the FPGA is programmed.

Part E - Reset branch (lines 77-88)

Lines 77-88

```
    always @(posedge clk) begin
        if (reset) begin
            st          <= C_RST;
            wcnt        <= 20'd0;
            q           <= 2'd0;
            idx         <= 5'd0;
            done        <= 1'b0;
            cam_reset_n <= 1'b0;
            sclk        <= 1'b1;
            sda_o       <= 1'b1;
            exposure    <= ({12'd0, exposure_sel} + 16'd1) << 8;   // 256 .. 4096 rows
        end else begin
```

- On KEY0 everything restarts from C_RST, and SCLK and SDATA go high (the idle state of an I2C bus).
- **exposure = (exposure_sel + 1) << 8**: switches 0000 give 1 x 256 = 256 rows, 1111 give 16 x 256 = 4096 rows. **<< 8** multiplies by 256.

Part F - Camera reset and start-up wait (lines 91-101)

Lines 91-101

```
            C_RST: begin                                  // camera in reset for 2 ms
                cam_reset_n <= 1'b0;
                wcnt <= wcnt + 20'd1;
                if (wcnt == 20'd50000) begin wcnt <= 20'd0; st <= C_WAIT; end
            end

            C_WAIT: begin                                 // release reset, wait 20 ms
                cam_reset_n <= 1'b1;
                wcnt <= wcnt + 20'd1;
                if (wcnt == 20'd500000) begin q <= 2'd0; st <= C_START; end
            end
```

- **C_RST**: hold cam_reset_n = 0 for 50,000 clocks = 2 ms at 25 MHz.
- **C_WAIT**: release reset and wait 500,000 clocks = 20 ms so the sensor is ready to accept settings, then start I2C with q = 0.

Part G - I2C START condition (lines 103-116)

Lines 103-116

```
            C_START: if (tick) begin                      // START: SDA falls while SCL high
                case (q)
                    2'd0: begin sda_o <= 1'b1; sclk <= 1'b1; end
                    2'd1: sda_o <= 1'b0;
                    2'd2: sclk  <= 1'b0;
                    2'd3: begin
                        sh     <= {8'hBA, entry(idx, exposure)};
                        byte_i <= 2'd0;
                        bit_i  <= 4'd0;
                        st     <= C_BIT;
                    end
                endcase
                q <= q + 2'd1;
            end
```

- An I2C transfer begins when **SDATA falls while SCLK is high**. That is quarters 0 and 1.
- Quarter 2 pulls SCLK low, ready for the first bit.
- Quarter 3 loads **sh = {8'hBA, entry(idx, exposure)}**: 0xBA is the camera's I2C write address, followed by the register address and 16-bit value, 4 bytes = 32 bits in total.
- **q <= q + 1** is a 2-bit counter, so after 3 it wraps to 0 automatically.

Part H - Sending bits and acknowledge slots (lines 118-139)

Lines 118-139

```
            C_BIT: if (tick) begin                        // one data bit or the ACK slot
                case (q)
                    2'd0: begin
                        sclk  <= 1'b0;
                        sda_o <= (bit_i == 4'd8) ? 1'b1 : sh[31];   // release SDA for ACK
                    end
                    2'd1: sclk <= 1'b1;
                    2'd2: sclk <= 1'b1;
                    2'd3: begin
                        sclk <= 1'b0;
                        if (bit_i == 4'd8) begin
                            bit_i <= 4'd0;
                            if (byte_i == 2'd3) st <= C_STOP;
                            else                byte_i <= byte_i + 2'd1;
                        end else begin
                            sh    <= {sh[30:0], 1'b0};
                            bit_i <= bit_i + 4'd1;
                        end
                    end
                endcase
                q <= q + 2'd1;
            end
```

- Quarter 0: SCLK low, put the next bit on SDATA (data may only change while SCLK is low).
- Quarters 1-2: SCLK high; the camera reads the bit during this time.
- Quarter 3: SCLK low again, shift **sh** left by one so the next bit is on top.
- After every 8 data bits comes slot 8, the **acknowledge**: the FPGA releases SDATA and the camera pulls it low to say 'received'. Our code does not check the answer, which keeps it simple.
- 4 bytes x 9 slots = 36 bit slots per register write. After the 4th byte, go to C_STOP.

Part I - STOP condition, next register, done (lines 141-161)

Lines 141-161

```
            C_STOP: if (tick) begin                       // STOP: SDA rises while SCL high
                case (q)
                    2'd0: begin sclk <= 1'b0; sda_o <= 1'b0; end
                    2'd1: sclk  <= 1'b1;
                    2'd2: sda_o <= 1'b1;
                    2'd3: begin
                        if (idx == NREG - 1) st <= C_DONE;
                        else begin idx <= idx + 5'd1; st <= C_START; end
                    end
                endcase
                q <= q + 2'd1;
            end

            C_DONE: done <= 1'b1;

            default: st <= C_RST;
            endcase
        end
    end

endmodule
```

- A transfer ends when **SDATA rises while SCLK is high** (quarters 1-2).
- Then **idx** moves to the next register and goes back to C_START, until all 15 (idx = 14) are sent.
- **C_DONE** keeps done = 1. One register write takes 44 bit times (start + 36 + stop + spare) = about 0.44 ms; 15 writes take about 6.7 ms.

3. Questions the professor may ask

Q: What is I2C?

A two-wire serial bus: SCLK carries the clock from the master (our FPGA), SDATA carries the data both ways. Many devices can share it; each has an address (the camera's write address is 0xBA).

Q: Why is SDATA open-drain?

Both the FPGA and the camera need to drive it (the camera answers with an acknowledge). With open drain each side can only pull it low, so two drivers never fight.

Q: Why did you write your own I2C master instead of using an IP?

The project rule was Verilog only, and a write-only master is small: a quarter-bit timer, a shift register and 6 states.

Q: What happens if the camera does not acknowledge?

Our code ignores the acknowledge, so it still finishes. If the camera were missing, LEDG1 would still light but no frames would arrive (LEDG2 would stay on).

Q: How do you get 640 x 640 from a 5 MP sensor?

Registers 0x01-0x04 choose a 1280 x 1280 window in the middle, and 0x22/0x23 skip every second row and column, which halves both sizes.

Q: Why mirror rows and columns (0x20)?

The camera is mounted so the raw picture is upside down; mirroring both directions rotates it by 180 degrees.

Q: How does exposure change brightness?

Register 0x09 sets how many row-times each pixel collects light. Longer exposure gives a brighter picture but more motion blur and fewer frames per second.